

Modul Praktikum: CRUD Data Poli dengan React Native

Mata Kuliah: Pengembangan Aplikasi Mobile | **Sub-CPMK:** 5, 6, 7 (CPMK3 & CPMK4)

Studi Kasus: Frontend SIMRS: pengelolaan master data **poli** (poliklinik)

Estimasi waktu: 2 × 50 menit

1. Tujuan Praktikum

Setelah praktikum, mahasiswa mampu:

1. Membangun aplikasi React Native (Expo) dengan navigasi antar layar.
2. Menghubungkan aplikasi ke RESTful API PHP menggunakan **fetch**.
3. Mengimplementasikan operasi **CRUD** (Create, Read, Update, Delete) serta validasi form.
4. Menjalankan aplikasi yang sama di **browser (web)** maupun perangkat mobile dengan kode lintas platform.

2. Dasar Teori Singkat

- **React Native:** framework JavaScript untuk membuat aplikasi Android/iOS dari satu basis kode (analogi: satu resep, dua jenis hidangan).
- **Expo:** perangkat bantu yang menyederhanakan pembuatan, menjalankan, dan menguji proyek React Native.
- **State (`useState`):** data yang bila berubah akan membuat tampilan diperbarui otomatis.
- **REST API:** "pelayan restoran" antara aplikasi (pelanggan) dan database (dapur); aplikasi meminta data lewat URL.
- **Studi kasus industri:** aplikasi pendaftaran pasien rumah sakit menampilkan daftar poli (Umum, Gigi, Anak) dari server agar pasien dapat memilih tujuan kunjungan.

3. Ringkasan Endpoint API

Basis URL: **http://localhost/api-simrs**

Operasi	Endpoint	Method	Parameter
Read (semua)	/poli/index.php	GET	-
Read (cari nama)	/poli/index.php?id=%kata%	GET	id diisi pola LIKE (hanya 1 baris dikembalikan)
Create	/poli/create.php	POST	JSON body: nama_poli , lokasi
Update	/poli/update.php	POST	Query string: id , nama_poli , lokasi
Delete	/poli/delete.php	POST	Query string: id

Format respons (dari **sendResponse**): { "status": "success|error", "message": "...", "data": ... }

Catatan penting: **update.php** dan **delete.php** membaca **\$_GET**, sehingga parameter dikirim lewat URL, bukan body JSON. Kode di modul ini menyesuaikan hal tersebut.

4. Alat dan Bahan

Node.js LTS, VS Code, XAMPP (Apache + MySQL aktif, folder **api-simrs** di **htdocs**), browser (Chrome/Edge) untuk pengujian web, serta aplikasi **Expo Go** di ponsel atau emulator Android Studio (opsional).

5. Langkah Kerja

Langkah 1 — Uji API

Buka <http://localhost/api-simrs/poli/index.php> di browser. Pastikan muncul JSON berisi daftar poli. Bila gagal, periksa XAMPP dan basis data sebelum lanjut.

Langkah 2 — Membuat Proyek

```
npx create-expo-app@latest simrs-poli --template blank
cd simrs-poli
npm install @react-navigation/native @react-navigation/native-stack
npx expo install react-native-screens react-native-safe-area-context
# Dependensi agar dapat berjalan di browser
npx expo install react-dom react-native-web @expo/metro-runtime
```

Langkah 3 — Struktur Folder

```
simrs-poli/
├── App.js
├── src/
│   ├── config/api.js
│   ├── services/poliService.js
│   ├── utils/dialog.js
│   └── screens/
│       ├── PoliListScreen.js
│       └── PoliFormScreen.js
```

Buat folder **src/config**, **src/services**, **src/utils**, **src/screens**.

Langkah 4 — Konfigurasi URL (src/config/api.js)

localhost pada emulator/ponsel menunjuk ke perangkat itu sendiri, bukan komputer Anda. Karena itu alamatnya harus disesuaikan. Pada **browser**, **localhost** dapat dipakai karena browser berjalan di komputer yang sama dengan XAMPP (**Platform.OS** bernilai **'web'**).

```
import { Platform } from 'react-native';

// Ganti 192.168.1.10 dengan IP LAN komputer Anda (cek: ipconfig / ifconfig)
const LAN_IP = '192.168.1.10';
const USE_REAL_DEVICE = false; // true hanya bila memakai Expo Go di ponsel fisik

const HOST = USE_REAL_DEVICE
  ? LAN_IP
  : Platform.OS === 'android' ? '10.0.2.2' : 'localhost';

// Bila Apache memakai port lain (mis. 8080), tulis: `http://${HOST}:8080/api-simrs`
export const BASE_URL = `http://${HOST}/api-simrs`;
```

Untuk pengujian di browser, biarkan `USE_REAL_DEVICE = false`. Ubah ke `true` hanya saat memakai ponsel fisik (ponsel dan komputer harus satu jaringan Wi-Fi).

Langkah 5 — Lapisan Layanan API (`src/services/poliService.js`)

Semua pemanggilan API dikumpulkan di satu berkas agar layar tetap sederhana (prinsip **separation of concerns**).

```
import { BASE_URL } from '../config/api';

async function request(url, options) {
  const res = await fetch(url, options);
  const json = await res.json();
  if (!res.ok || json.status !== 'success') {
    throw new Error(json.message || 'Terjadi kesalahan pada server');
  }
  return json;
}

const q = encodeURIComponent;

// READ - semua poli
export const getPoli = async () =>
  (await request(`${BASE_URL}/poli/index.php`)).data;

// READ - cari berdasarkan nama (API mengembalikan 1 objek)
export const searchPoli = async (kata) =>
  (await request(`${BASE_URL}/poli/index.php?id=${q('%' + kata + '%')}`)).data;

// CREATE - body JSON
export const createPoli = (nama_poli, lokasi) =>
  request(`${BASE_URL}/poli/create.php`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ nama_poli, lokasi }),
  });

// UPDATE - parameter lewat query string
export const updatePoli = (id, nama_poli, lokasi) =>
  request(
    `${BASE_URL}/poli/update.php?id=${id}&nama_poli=${q(nama_poli)}&lokasi=${q(lokasi)}`,
    { method: 'POST' }
  );

// DELETE - parameter lewat query string
export const deletePoli = (id) =>
  request(`${BASE_URL}/poli/delete.php?id=${id}`, { method: 'POST' });
```

Langkah 6 — Helper Dialog Lintas Platform (`src/utils/dialog.js`)

`Alert.alert` bawaan React Native **tidak berfungsi di browser**: dialog tidak muncul dan tombolnya tidak pernah menjalankan aksi. Karena itu dibuat helper yang memilih cara tampil sesuai platform: `window.alert/window.confirm` di web, dan `Alert.alert` di Android/iOS.

```
import { Alert, Platform } from 'react-native';

// Pesan informasi; onOk dijalankan setelah pengguna menekan OK
export function showAlert(title, message, onOk) {
  if (Platform.OS === 'web') {
    window.alert(`${title}\n\n${message}`);
    if (onOk) onOk();
  } else {
    Alert.alert(title, message, [{ text: 'OK', onPress: onOk }]);
  }
}
```

```

    }
  }

  // Dialog konfirmasi; onConfirm dijalankan bila pengguna menyetujui
  export function confirmDialog(title, message, onConfirm) {
    if (Platform.OS === 'web') {
      if (window.confirm(`${title}\n\n${message}`)) onConfirm();
    } else {
      Alert.alert(title, message, [
        { text: 'Batal', style: 'cancel' },
        { text: 'Hapus', style: 'destructive', onPress: onConfirm },
      ]);
    }
  }
}

```

Langkah 7 — Layar Daftar Poli (src/screens/PoliListScreen.js)

Fitur: menampilkan daftar (**Read**), pencarian, tarik-untuk-muat-ulang (di mobile), tombol edit, hapus dengan konfirmasi (**Delete**). Dialog memakai helper `showAlert` dan `confirmDialog` dari Langkah 6, bukan `Alert` langsung.

```

import React, { useCallback, useState } from 'react';
import {
  View, Text, FlatList, TextInput, TouchableOpacity,
  StyleSheet, ActivityIndicator,
} from 'react-native';
import { useFocusEffect } from '@react-navigation/native';
import { getPoli, searchPoli, deletePoli } from '../services/poliService';
import { showAlert, confirmDialog } from '../utils/dialog';

export default function PoliListScreen({ navigation }) {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(false);
  const [keyword, setKeyword] = useState('');

  const loadData = async () => {
    setLoading(true);
    try {
      setData(await getPoli());
    } catch (e) {
      showAlert('Gagal memuat', e.message);
    } finally {
      setLoading(false);
    }
  };

  // Dipanggil setiap layar ini tampil kembali (mis. setelah simpan data)
  useFocusEffect(useCallback(() => { loadData(); }, []));

  const handleSearch = async () => {
    if (keyword.trim() === '') return loadData();
    setLoading(true);
    try {
      const hasil = await searchPoli(keyword.trim());
      setData([hasil]); // API mengembalikan satu objek
    } catch (e) {
      setData([]); // 404 = tidak ditemukan
    } finally {
      setLoading(false);
    }
  };

  const handleDelete = (item) => {
    confirmDialog('Hapus Poli', `Yakin menghapus "${item.nama_poli}"?`, async () => {
      try {

```

```

        await deletePoli(item.id);
        loadData();
    } catch (e) {
        showAlert('Gagal menghapus', e.message);
    }
  });
};

const renderItem = ({ item }) => (
  <View style={s.card}>
    <View style={{ flex: 1 }}>
      <Text style={s.title}>{item.nama_poli}</Text>
      <Text style={s.sub}>📍 {item.lokasi || '-'}</Text>
    </View>
    <TouchableOpacity
      style={[s.btn, { backgroundColor: '#f59e0b' }]}
      onPress={() => navigation.navigate('PoliForm', { poli: item })}>
      <Text style={s.btnText}>Edit</Text>
    </TouchableOpacity>
    <TouchableOpacity
      style={[s.btn, { backgroundColor: '#ef4444' }]}
      onPress={() => handleDelete(item)}>
      <Text style={s.btnText}>Hapus</Text>
    </TouchableOpacity>
  </View>
);

return (
  <View style={s.container}>
    <View style={s.searchRow}>
      <TextInput
        style={s.input}
        placeholder="Cari nama poli..."
        value={keyword}
        onChangeText={setKeyword}
        onSubmitEditing={handleSearch}
      />
      <TouchableOpacity style={[s.btn, { backgroundColor: '#2563eb' }]} onPress={handleSearch}>
        <Text style={s.btnText}>Cari</Text>
      </TouchableOpacity>
    </View>

    {loading && <ActivityIndicator size="large" color="#2563eb" />}

    <FlatList
      data={data}
      keyExtractor={(item) => String(item.id)}
      renderItem={renderItem}
      refreshing={loading}
      onRefresh={loadData}
      ListEmptyComponent={!loading && <Text style={s.empty}>Data poli tidak ditemukan</Text>}
    />

    <TouchableOpacity style={s.fab} onPress={() => navigation.navigate('PoliForm')}>
      <Text style={s.fabText}>+</Text>
    </TouchableOpacity>
  </View>
);
}

const s = StyleSheet.create({
  container: { flex: 1, padding: 12, backgroundColor: '#f3f4f6' },
  searchRow: { flexDirection: 'row', marginBottom: 10 },
  input: {
    flex: 1, backgroundColor: 'fff', borderRadius: 8,
    paddingHorizontal: 12, marginRight: 8, borderWidth: 1, borderColor: '#d1d5db',
  },
  card: {
    flexDirection: 'row', alignItems: 'center', backgroundColor: 'fff',

```

```

padding: 14, borderRadius: 10, marginBottom: 10, elevation: 2,
},
title: { fontSize: 16, fontWeight: 'bold' },
sub: { color: '#6b7280', marginTop: 2 },
btn: { paddingVertical: 8, paddingHorizontal: 12, borderRadius: 6, marginLeft: 6 },
btnText: { color: 'fff', fontWeight: '600' },
empty: { textAlign: 'center', marginTop: 30, color: '#6b7280' },
fab: {
  position: 'absolute', right: 20, bottom: 24, width: 56, height: 56,
  borderRadius: 28, backgroundColor: '#16a34a', alignItems: 'center',
  justifyContent: 'center', elevation: 5,
},
fabText: { color: 'fff', fontSize: 28, marginTop: -2 },
});

```

Langkah 8 — Layar Form Tambah/Ubah (src/screens/PoliFormScreen.js)

Satu layar dipakai untuk **Create** dan **Update**. Mode ditentukan oleh ada tidaknya parameter **poli**.

```

import React, { useState } from 'react';
import {
  View, Text, TextInput, TouchableOpacity, StyleSheet, ActivityIndicator,
} from 'react-native';
import { createPoli, updatePoli } from '../services/poliService';
import { showAlert } from '../utils/dialog';

export default function PoliFormScreen({ route, navigation }) {
  const poli = route.params?.poli; // undefined = mode tambah
  const isEdit = !!poli;

  const [namaPoli, setNamaPoli] = useState(poli?.nama_poli ?? '');
  const [lokasi, setLokasi] = useState(poli?.lokasi ?? '');
  const [error, setError] = useState('');
  const [saving, setSaving] = useState(false);

  const handleSave = async () => {
    // Validasi input (Sub-CPMK 6)
    if (namaPoli.trim() === '') {
      setError('Nama poli wajib diisi');
      return;
    }
    setError('');
    setSaving(true);
    try {
      const res = isEdit
        ? await updatePoli(poli.id, namaPoli.trim(), lokasi.trim())
        : await createPoli(namaPoli.trim(), lokasi.trim());
      // goBack dijalankan setelah OK ditekan (berlaku di web maupun mobile)
      showAlert('Berhasil', res.message, () => navigation.goBack());
    } catch (e) {
      showAlert('Gagal menyimpan', e.message);
    } finally {
      setSaving(false);
    }
  };

  return (
    <View style={s.container}>
      <Text style={s.label}>Nama Poli *</Text>
      <TextInput
        style={[s.input, error && s.inputError]}
        placeholder="Contoh: Poli Gigi"
        value={namaPoli}
        onChangeText={setNamaPoli}
      />
      <!!error && <Text style={s.error}>{error}</Text>
    </View>
  );
}

```

```

<Text style={s.label}>Lokasi</Text>
<TextInput
  style={s.input}
  placeholder="Contoh: Gedung A Lantai 1"
  value={lokasi}
  onChangeText={setLokasi}
/>

<TouchableOpacity style={s.btn} onPress={handleSave} disabled={saving}>
  {saving
    ? <ActivityIndicator color="#fff" />
    : <Text style={s.btnText}>{isEdit ? 'Perbarui' : 'Simpan'}</Text>}
</TouchableOpacity>
</View>
);
}

const s = StyleSheet.create({
  container: { flex: 1, padding: 16, backgroundColor: '#fff' },
  label: { fontWeight: '600', marginTop: 14, marginBottom: 6 },
  input: { borderWidth: 1, borderColor: '#d1d5db', borderRadius: 8, padding: 10 },
  inputError: { borderColor: '#ef4444' },
  error: { color: '#ef4444', marginTop: 4 },
  btn: { backgroundColor: '#2563eb', padding: 14, borderRadius: 8, marginTop: 24, alignItems: 'center' },
  btnText: { color: '#fff', fontWeight: 'bold', fontSize: 16 },
});

```

Langkah 9 — Navigasi (App.js)

```

import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import PoliListScreen from './src/screens/PoliListScreen';
import PoliFormScreen from './src/screens/PoliFormScreen';

const Stack = createNativeStackNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="PoliList" component={PoliListScreen} options={{ title: 'Data Poli' }} />
        <Stack.Screen
          name="PoliForm"
          component={PoliFormScreen}
          options={({ route }) => ({
            title: route.params?.poli ? 'Ubah Poli' : 'Tambah Poli',
          })}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}

```

Langkah 10 — Menjalankan Aplikasi

A. Di browser (disarankan untuk praktikum)

```
npx expo start --web
```

Atau jalankan `npx expo start`, lalu tekan `w`. Browser terbuka di `http://localhost:8081` (port dapat berbeda, lihat pesan di terminal). Tekan **F12** untuk membuka DevTools: tab **Network** menampilkan request dan respons API, tab **Console** menampilkan error JavaScript.

B. Di ponsel atau emulator

```
npx expo start
```

Pindai kode QR dengan Expo Go (ponsel), atau tekan `a` untuk emulator Android. Pastikan `USE_REAL_DEVICE` di `api.js` sudah sesuai.

Catatan web: gestur tarik-untuk-muat-ulang (`onRefresh`) tidak tersedia di browser; gunakan tombol refresh browser atau tombol Cari dengan kolom kosong untuk memuat ulang data.

6. Skenario Pengujian

No	Skenario	Hasil yang diharapkan
1	Buka aplikasi	Daftar poli tampil
2	Tambah poli dengan nama kosong	Pesan "Nama poli wajib diisi"
3	Tambah "Poli Mata"	Alert berhasil, data muncul di daftar
4	Edit lokasi "Poli Mata"	Data berubah setelah kembali ke daftar
5	Hapus "Poli Mata"	Dialog konfirmasi, lalu data hilang
6	Cari "Gigi"	Poli yang cocok tampil
7	Cari kata yang tidak ada	"Data poli tidak ditemukan"
8	(Browser) Tekan Hapus lalu batalkan dialog	Data tidak terhapus
9	(Browser) Simpan data, tekan OK pada dialog	Kembali ke daftar, data baru tampil
10	(Browser) Buka DevTools > Network saat menyimpan	Request <code>create.php</code> berstatus 201

7. Pemecahan Masalah

Gejala	Penyebab	Solusi
<code>Network request failed</code>	Memakai <code>localhost</code> di perangkat	Gunakan IP LAN (ponsel) atau <code>10.0.2.2</code> (emulator Android)
Tetap gagal terhubung	Firewall memblokir Apache	Izinkan port 80 pada firewall
<code>JSON Parse error</code>	Server mengembalikan error PHP berbentuk HTML	Buka URL di browser untuk melihat pesan error
Edit tidak mengubah data	Parameter dikirim lewat body JSON	Gunakan query string (lihat Langkah 5)
<code>blocked by CORS policy</code> (browser)	Error PHP muncul sebelum header CORS terkirim	Buka URL API langsung di browser, perbaiki error PHP, pastikan tidak ada spasi sebelum <code><?php</code>
Tombol Hapus tidak bereaksi (browser)	Masih memakai <code>Alert.alert</code>	Gunakan <code>confirmDialog</code> (Langkah 6 dan 7)
Setelah Simpan layar tidak kembali (browser)	Callback <code>Alert.alert</code> tidak berjalan di web	Gunakan <code>showAlert</code> dengan callback (Langkah 8)

Gejala	Penyebab	Solusi
Halaman web kosong	Dependensi web belum terpasang	Jalankan perintah Langkah 2, lalu <code>npx expo start --web --clear</code>
<code>Failed to fetch</code> (browser)	Apache mati atau port pada <code>BASE_URL</code> salah	Periksa XAMPP dan <code>BASE_URL</code>
Pencarian hanya menampilkan 1 hasil	<code>index.php</code> memakai <code>fetch()</code>	Keterbatasan API; ubah ke <code>fetchAll()</code> bila diperlukan

8. Catatan Kualitas API (Bahan Diskusi)

- `update.php` dan `delete.php` memakai `$_GET`: operasi mengubah data sebaiknya via body (POST/PUT/DELETE), karena URL tercatat di log dan mudah terpicu tidak sengaja.
- Update dan delete mengembalikan kode `201` (Created); kode yang lebih tepat adalah `200`.
- `Access-Control-Allow-Origin: *` cocok untuk praktikum, tetapi tidak aman untuk produksi.
- Belum ada autentikasi `api_token` (tabel `users`) pada endpoint poli.

9. Tugas Mandiri

- Terapkan pola yang sama untuk tabel **dokter** (`nama`, `spesialisasi`, `no_sip`).
- Tambahkan validasi: nama poli minimal 3 karakter.
- Tambahkan indikator "Tidak ada koneksi" saat server tidak dapat dijangkau.

Rubrik penilaian: Read dan navigasi (25%), Create + validasi (25%), Update (20%), Delete + konfirmasi (15%), Kerapian kode dan struktur (15%).

10. Referensi

- React Native Team. (2024). *React Native Documentation*. <https://reactnative.dev/docs/getting-started>
- Expo. (2024). *Expo Documentation*. <https://docs.expo.dev>
- React Navigation. (2024). *Native Stack Navigator*. <https://reactnavigation.org/docs/native-stack-navigator>
- MDN Web Docs. (2024). *Using the Fetch API*. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- Eisenman, B. (2016). *Learning React Native* (2nd ed.). O'Reilly Media.